

1. Analizar la ISA: memoria-registros vs registro-registro, unidades direccionables (diferencias entre microOrgaSmall y RiscV).

En MicroOrgaSmall usa memoria-registro, osea las instrucciones pueden operar entre la memoria y los registros, por ejemplo: LOAD R1 [M], STR [M] R2, etc.

A cambio RiscV usa registro-registro, solo operamos entre registros mayormente, porque para usar memoria primero usamos lw, lh, etc y luego add, sub, etc.

Con respecto a las unidades direccionables,

en RiscV usamos lb(carga byte), lh(carga media palabra), lw(cargamos palabra entera), sb(guardo byte) etc.

en MicroOrgaSmall no hay particiones de byte, half o word, solo usa 1 byte(8 bits).

2. ¿Cómo se diferencian los registros y las instrucciones que utilizan registros entre las arquitecturas MicroOrgaSmall y RISC-V?

MicroOrgaSmall tiene 8 registros de 8 bits y sus instrucciones con registros siempre operan con dos registros (Rx y Ry).

RiscV tiene 32 registros de 32 bits y sus instrucciones utilizan 3 registros (rd, rs1 y rs2).

3. ¿Qué implicancias tiene esa diferencia en la cantidad de accesos a memoria que se realizan en la etapa de fetch? ¿Cómo se hace una lectura del PC?

Las implicancias para microOrgaSmall es que, como utiliza memoria de 8 bits y sus instrucciones son de 16 bits, su etapa fetch necesita 2 accesos a memoria para el fetch.

Y las implicancias para RiscV son mejores porque usa instrucciones de 32 bits y su memoria es de 32 bits, así que basta con un único acceso para el fetch.

Con respecto a la lectura del PC,

En microOrgaSmall, el PC debe realizar dos accesos consecutivos a memoria durante el fetch porque las instrucciones son de 16 bits y el bus es de 8 bits y la memoria también, por lo cual cada acceso solo trae 1 byte por eso se lee la instrucción en dos partes (IR-L y IR-H). Y como se lee a byte **el PC se incrementa de a 1** para pasar del primer byte al segundo.

En RiscV, el PC con un único acceso basta ya que sus palabras son de 32 bits y sus instrucciones también son de 32 bits (4 bytes), y como sus instrucciones son de 4 bytes **el PC se incrementa + 4**.

4. ¿Qué tamaño o tipo de unidades direccionables usa cada arquitectura (por byte, por palabra, etc.) y cómo afecta eso el cálculo de direcciones?

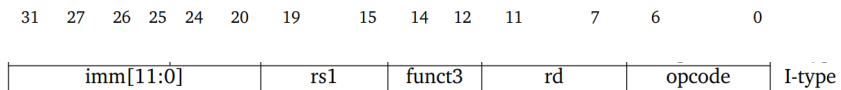
En microOrgaSmall direcciona por byte y cada acceso lee o escribe solo 1 byte. lo que implica que el PC avance de a 1 y haya fetch doble para sus instrucciones de 16 bits.

En cambio RiscV también direcciona por byte solo que también puede acceder a los bytes usando lh(2bytes), lw(4bytes), lb(1byte) porque las

instrucciones ocupan 4bytes, lo que hace que el PC avance de a 4 bytes.

5. ¿Cómo se decodifica una instrucción tipo R en RISC-V? Por ejemplo ADDI t0, t1, 0x765

ADDI es de tipo I no R,



opcode = 0010011 (para tipo I), 7 bits

funct3 = 000 (suma con inmediato - me dice que operación de las de tipo I usar) , 3 bits

imm[11:0] = 0x765 (inmediato de 12 bits)

rd = t0 (5 bits)

rs1 = t1 (5 bits)

32 bits en total.

Primero el hardware durante la decodificación lee el opcode(para saber el tipo de instruccion), luego lee funct3(para saber el tipo de opracion), luego extrae rd y rs1 y el inmediato(imm[11:0]) para luego ejecutarlo (rd = rs1 + imm) y generar señales de control hacia la ALU, banco de registros y unidad de memoria.

6. ¿Qué pasaría si el inmediato tuviese más de 12 bits, por ejemplo ADDI t0, t1, 0xAFF3

La instrucción ADDI solo acepta inmediatos de 12 bits por lo cual a priori no se podria decodificar pero en RiscV puedes usar combinaciones como: lui(20 bits mas significativos) + addi(12 bits menos significativos) para formar un inmediato mayor a 12bits.

7. ¿Por qué en la arquitectura MicroOrgaSmall tengo la instrucción MOV para mover valores entre registros y en RISC-V no?

MicroOrgaSmall necesita la instrucción MOV para copiar entre registros, porque no tengo un registro constante cero como en RiscV.

En cambio, RISC-V no incluye MOV porque la operación de copia puede realizarse mediante ADDI rd, rs1, 0, aprovechando que el registro x0 siempre vale cero.

8. Mencionar las diferencias entre los tipos de instrucciones de salto condicional e incondicional entre MicroOrgaSmall y RISC-V

La diferencia de los saltos es que en MicroOrgaSmall son a una dirección absoluta (JC, JZ, JN y JMP), mientras que en Risc siempre se suman inmediatos (beq, bne, blt, bge, jal, etc).

9. En RISC-V, ¿los saltos condicionales son absolutos o relativos? ¿Y en MicroOrgaSmall? Comparar los mecanismos que usan RISC-V y MicroOrgaSmall para determinar si debe realizar un salto condicional.

En RiscV los saltos condicionales son relativos, por que todos los saltos (beq, bne, blt, etc) actualizan el PC sumando un “imm”(offset) sumados a la posición actual.

Y en MicroOrgaSmall los saltos condicionales son absolutos, lee inmediatos y los establece al PC.

Con respecto a los mecanismos que usan para determinar si se debe realizar un salto condicional,

En RiscV se compara entre registros.

En MicroOrgaSmall utiliza los flags de la ALU(N, Z, C) en el instante que se ejecuta la instrucción para hacer un jump.

10. ¿Cuál es la ventaja de tener operaciones de tres registros en RISC-V en lugar de dos como MicroOrgaSmall?

Las operaciones con un tercer registro permiten evitar mover datos entre registros, por ejemplo, en MicroOrgaSmall cada vez que hacemos una suma sobreescribimos R1 y si queremos conservar su valor tendríamos que copiarlo primero, por lo que la programación es mas fácil (aunque complejiza las instrucciones).